

Allow static data members in local and unnamed classes

Document #: P3588R0
Date: 2025-01-13
Project: Programming Language C++
Audience: EWG
Reply-to: Brian Bi
<bbi10@bloomberg.net>

Contents

- 1 [Abstract](#)
- 2 [Background](#)
- 3 [Why this is useful](#)
- 4 [Proposal](#)
- 5 [Implementation experience](#)
- 6 [Proposed wording](#)
- 7 [References](#)

§ 1 Abstract

Local and unnamed classes (as well as classes nested within) are not permitted to declare static data members. This restriction dates back to C++98, when there was no way to provide a definition for such a member. In modern C++, static data members can have inline definitions, so this rationale is obsolete. Static data members can be useful in local classes for the same reasons why they are useful in non-local classes, so this paper proposes to allow them in C++26.

§ 2 Background

C++03 §[`class.static.data`] stated as follows:

[...] Unnamed classes and classes contained directly or indirectly within unnamed classes shall not contain `static` data members. [*Note: this is because there is no mechanism to provide the definitions for such `static` data members.*]

[...] A local class shall not have `static` data members.

No rationale is explicitly provided for the latter restriction. However, we can see that “there is no mechanism to provide the definitions” applies to static data members of local classes as well. There was one additional possible issue, which is the need to generate an external symbol based on the identity of the local class, but it seems that this applies equally well to functions (*i.e.*, it seems that a member function of a local class in an inline function would need to have a weak external symbol since its address could be taken), and in any case, thanks to [N2657], unnamed and local classes became valid template arguments from C++11 and it is certain that all implementations can generate appropriate mangled names for local classes and members of local and unnamed classes. There do not appear to be any remaining technical obstacles to allowing such classes to have static data members.

[CWG728] proposed to relax some restrictions on local classes; more specifically, to allow them to be templates, declare member templates, declare friends, and declare static data members. It was determined that this was a feature request, and a paper was submitted, [P2044R2]. However, that paper addressed only the issue of member templates. *This* paper deals only with the simpler feature of static data members.

§ 3 Why this is useful

C++ entities should generally be declared in the narrowest scope in which they are needed, which may be a single function. For example, in Google Test, the `TEST` macro is used to define a single function which then verifies some expected properties. If a class is required only for that single test, it is convenient to declare it as a local class. It can be desirable for such a local class to have a static data member, such as a static data member that tracks the number of live objects of the class.

Here is another example: suppose that a C++ library defines a custom version of the tuple protocol in which a tuple-like class is expected to declare a static data member named `tuple_size`, initialized to a compile-time constant. If a unit test needs to declare such a class and pass it as a template argument to the component under test (which is expecting it to be a tuple), it would be convenient to be able to scope that class to the smallest enclosing block.

In the former case, the static data member would be declared `static inline`. In the latter case, it would be declared `static constexpr`, which makes it implicitly inline. A non-inline static data member in a local class cannot be defined, but I see no reason not to also allow it. It could be used just for its type (*i.e.*, in unevaluated contexts only). Allowing such a variable to inhabit class scope would make it more narrowly scoped than if it were a local `extern` variable inhabiting the enclosing function.

§ 4 Proposal

Allow static data members to be declared by local and unnamed classes without restriction, except unnamed classes that have a typedef name for linkage purposes. That is, the following would remain ill formed:

```
typedef struct {  
    inline static int x = 0;  
} S;
```

§9.2.4 [dcl.typedef]¹p5 places various restrictions on such classes in the vein of requiring them to be “C-like”: for example, they can’t have member functions nor member typedefs. Since static data members don’t exist in C, I don’t propose to drop this restriction.

Static data members of local classes follow the same initialization order rules as if their classes were non-local: for example, if *x*, *y*, and *z* are three inline variables and in every translation unit they appear in that order, and *x* and *z* belong to non-local classes while *y* belongs to a local class, then *y* is initialized after *x* and before *z*, unless *y*’s enclosing function is templated, in which case *y* could be initialized in any order relative to *x* and *z*. I believe that this rule is intuitive and not difficult to implement.

§ 5 Implementation experience

GCC supports static data members in unnamed and local classes as an extension when `-fpermissive` is used. GCC does not allow such members to be explicitly declared `inline`, but this seems to be an oversight: the error message says “‘inline’ specifier invalid for variable ‘x’ declared at block scope”. GCC does allow the member to be `constexpr`, and generates an appropriate definition in that case (*i.e.*, the address can be taken).

I implemented this feature, including dynamic initialization of inline static data members, as a patch to Clang 19.1.4. The only changes required were as follows.

- Remove the restriction.
- Add a case for instantiation of templated variables: instantiate them eagerly when they’re static data members of local classes. (The Standard specifies that member of local classes are always instantiated eagerly.)
- When generating code for a function, iterate through its local classes as well, so that their static data members can be emitted.

The patch passes the Clang unit tests, but the feature has not been tested in a large code base.

§ 6 Proposed wording

Wording is relative to [N5001].

Modify §11.4.9.3 [class.static.data]² as follows.

A static data member shall not be `mutable` ([dcl.stc]). ~~A static data member shall not be a direct member ({class.mem}) of an unnamed ({class.pre}) or local ({class.local}) class or of a (possibly indirectly) nested class ({class.nest}) thereof.~~

§ 7 References

[CWG728] Faisal Vali. 2008-10-05. Restrictions on local classes.

<https://wg21.link/cwg728>

[N2657] John Spicer. 2008-06-10. Local and Unnamed Types as Template Arguments.

<https://wg21.link/n2657>

[N5001] Thomas Köppe. 2024-12-17. Working Draft, Programming Languages — C++.

<https://wg21.link/n5001>

[P2044R2] Robert Leahy. 2020-04-14. Member Templates for Local Classes.

<https://wg21.link/p2044r2>

-
1. All citations to the Standard are to working draft N5001 unless otherwise specified.↵